Year: 2026-2027 **Sem V-NEP**

Course: ASD&D Lab

Experiment 03

Date of Performance: 28/07/2026 **Date of Submission: 28/07/2026**

Class: T1

Student Roll No: 2401041 **Student Name:Lakshya Kanjwani**

LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Aim: To deploy multi-service applications using Docker Compose.

Theory:

Docker Compose is an orchestration tool that uses a single YAML file (docker-compose.yml) to define, configure, and run multi-container applications. It simplifies infrastructure management by handling services, networks, and persistent data in one deployment step.

- **Microservices Architecture:** Rather than building a single monolithic application, complex systems are divided into specialized, single-purpose services (e.g., a web server, database, and cache) that run in isolated but interconnected containers.
- **Port Mapping (ports)**: Bridging a container's internal, isolated port to an accessible port on the host machine. Customizing the host port allows you to bypass conflicts with services already running locally.
- **Data Persistence (volumes):** Containers are ephemeral; any data written inside them is lost when the container is removed. Volumes solve this by storing container data (like MySQL records) directly on the host machine, ensuring it survives restarts and rebuilds.
- **Startup Dependencies (depends_on):** Dictates the boot sequence of your infrastructure. This guarantees that prerequisite services, like a backend database, are fully running before dependent services, like a web interface, attempt to connect to them.

Application Architecture

The deployed environment is a four-tier microservice stack:

1. **Web Server (webserver):** An NGINX instance handling web traffic, mapped to the host machine on port 8080.
2. **Relational Database (database):** A MySQL 8.0 server equipped with a persistent volume (mysql-data) to protect data. It is mapped to host port 3307 to prevent conflicts with local MySQL installations.
3. **Caching Layer (rediscache):** A Redis in-memory data store running on port 6379 to accelerate data retrieval.
4. **Database Management (db-admin):** A PhpMyAdmin web interface providing a visual dashboard to manage the MySQL database.

Multi-Service Architecture Definition Steps:

1. Service Disaggregation (Microservices) Break the monolithic application down into isolated, single-purpose components (e.g., Web Server, Database, Cache, Management UI). Each component is defined as a distinct `service` within the `docker-compose.yml` file.

2. Image Definition Assign a specific, lightweight Docker image to each service (e.g., `nginx:latest`, `mysql:8.0`, `redis:alpine`). This ensures environment consistency across deployments.

3. Internal Network Establishment Rely on Docker Compose's default behavior to automatically create a private bridge network (e.g., `project_default`). This enables secure, internal DNS resolution where services communicate using their container names (e.g., `PMA_HOST: database`) rather than dynamic IP addresses.

4. Port Conflict Resolution (`ports`) Map internal container ports to custom host machine ports. By intentionally mapping to alternative host ports (e.g., `3307:3306` for MySQL, `8082:80` for the Admin UI), you prevent collisions with services already running on the physical host machine.

5. State Persistence (`volumes`) Configure Docker Volumes for stateful services (like databases). This maps the container's internal data directory to the host machine, guaranteeing that data survives container restarts, updates, or complete teardowns.

6. Dependency Management (`depends_on`) Enforce a strict startup sequence. By using `depends_on`, you dictate that foundational services (Database, Cache) must initialize completely before dependent services (Web Server, Admin UI) are permitted to start.

7. Environment Configuration (`environment`) Inject necessary credentials, flags, and configuration variables directly into the containers at runtime. This allows standard images to be customized for the specific application environment without requiring custom image builds.

8. Unified Deployment (`docker compose up`) Execute a single declarative command to orchestrate the creation of the network, the provisioning of volumes, the pulling of images, and the sequential startup of all defined containers as a cohesive stack.

Steps to implement:

1. **Initialize Project:**

```
C:\Users\15L>mkdir compose-demo

C:\Users\15L>cd compose-demo

C:\Users\15L\compose-demo>
C:\Users\15L\compose-demo>dir
 Volume in drive C is Windows
 Volume Serial Number is 4CC2-1CBF

 Directory of C:\Users\15L\compose-demo

07/28/2026  10:27 AM    <DIR>          .
07/28/2026  10:25 AM    <DIR>          ..
07/28/2026  10:27 AM               442 docker-compose.yml
               1 File(s)            442 bytes
               2 Dir(s)  760,263,254,016 bytes free
```

2. **Create Configuration:**

```yaml
version: '3.8'

services:
  web:
    image: nginx
    container_name: webserver
```

```
    ports:
      - "8080:80"

  database:
    image: mysql:8.0
    container_name: mysqlserver
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: studentdb
      MYSQL_USER: student
      MYSQL_PASSWORD: student123
    ports:
      - "3306:3306"
    volumes:
      - mysql-data:/var/lib/mysql

volumes:
  mysql-data:
```

3. **Verify and Deploy:**

```
dir
docker compose up -d
```

Phase 1: Inspect Running App

- Step 1: Check Container Status (docker compose ps)

```
C:\Users\15L\compose-demo>docker compose up -d
[+] Running 0/4
 - Network compose-demo_default      Created                                    0.2s
 - Volume "compose-demo_mysql-data"  Created                                    0.2s
 - Container mysqlserver             Starting                                   0.2s
 - Container webserver               Starting                                   0.2s
Error response from daemon: Ports are not available: exposing port TCP 0.0.0.0:3306 -> 0.0.0.0:0: list
en tcp 0.0.0.0:3306: bind: Only one usage of each socket address (protocol/network address/port) is no
rmally permitted.
```

```
C:\Users\15L\compose-demo>docker compose ps
NAME      IMAGE     COMMAND   SERVICE   CREATED   STATUS    PORTS
```

- Step 2: View Application Logs (docker compose logs)

```
C:\Users\15L\compose-demo>docker compose logs
```

- Step 3: List Local Docker Images (docker compose images)

```
C:\Users\15L\compose-demo>docker compose images
CONTAINER           REPOSITORY          TAG                 IMAGE ID            SIZE
mysqlserver         mysql               8.0                 6cd09145362d        799MB
webserver           nginx               latest              fd204fe2f750        161MB
```

- Step 4: Validate Configuration Syntax (docker compose config)

```
C:\Users\15L\compose-demo>docker compose config
name: compose-demo
services:
  database:
    container_name: mysqlserver
    environment:
      MYSQL_DATABASE: studentdb
      MYSQL_PASSWORD: student123
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_USER: student
    image: mysql:8.0
    networks:
      default: null
    ports:
      - mode: ingress
        target: 3306
        published: "3306"
        protocol: tcp
    volumes:
      - type: volume
        source: mysql-data
        target: /var/lib/mysql
        volume: {}
  web:
    container_name: webserver
    image: nginx
    networks:
      default: null
    ports:
      - mode: ingress
        target: 80
        published: "8080"
        protocol: tcp
networks:
  default:
    name: compose-demo_default
volumes:
  mysql-data:
    name: compose-demo_mysql-data
```

Phase 2: Pause and Resume

- Step 1: Gracefully Halt Services (docker compose stop)

```
C:\Users\15L\compose-demo>docker compose stop
[+] Stopping 2/0
 ✔Container webserver    Stopped
 ✔Container mysqlserver  Stopped
```

- Step 2: Resume Halted Services (docker compose start)

```
C:\Users\15L\compose-demo>docker compose start
[+] Running 0/0
 - Container webserver    Starting                                                    0.0s
 - Container mysqlserver  Starting                                                    0.0s
Error response from daemon: Ports are not available: exposing port TCP 0.0.0.0:8080 -> 0.0.0.0:0: list
en tcp 0.0.0.0:8080: bind: Only one usage of each socket address (protocol/network address/port) is no
rmally permitted.
```

- Step 3: Power Cycle Containers (docker compose restart)

```
C:\Users\15L\compose-demo>docker compose restart
[+] Restarting 2/2
 ✔Container webserver    Started                                                     0.6s
 ✔Container mysqlserver  Started                                                     0.6s
```

Phase 3: Foreground Mode

- Step 1: Teardown Infrastructure (docker compose down)

```
C:\Users\15L\compose-demo>docker compose down
[+] Running 3/3
 ✔Container mysqlserver         Removed                                             1.2s
 ✔Container webserver           Removed                                             0.4s
 ✔Network compose-demo_default  Removed                                             0.2s
```

- Step 2: Launch in Attached Mode (docker compose up)

```
C:\Users\15L\compose-demo>docker compose up
[+] Running 3/0
 ✔Network compose-demo_default  Created                                             0.0s
 ✔Container webserver           Created                                             0.0s
 ✔Container mysqlserver         Created                                             0.1s
Attaching to mysqlserver, webserver
Gracefully stopping... (press Ctrl+C again to force)
Error response from daemon: Ports are not available: exposing port TCP 0.0.0.0:3306 -> 0.0.0.0:0: list
en tcp 0.0.0.0:3306: bind: Only one usage of each socket address (protocol/network address/port) is no
rmally permitted.
```

Phase 4: The Clean Slate

- Step 1: Complete Teardown & Data Purge (docker compose down -v)

```
C:\Users\15L\compose-demo>docker compose down -v
[+] Running 4/4
 ✔Container webserver             Removed                                0.0s
 ✔Container mysqlserver           Removed                                0.0s
 ✔Volume compose-demo_mysql-data  Removed                                0.0s
 ✔Network compose-demo_default    Removed                                0.2s
```

Browser Verification Results

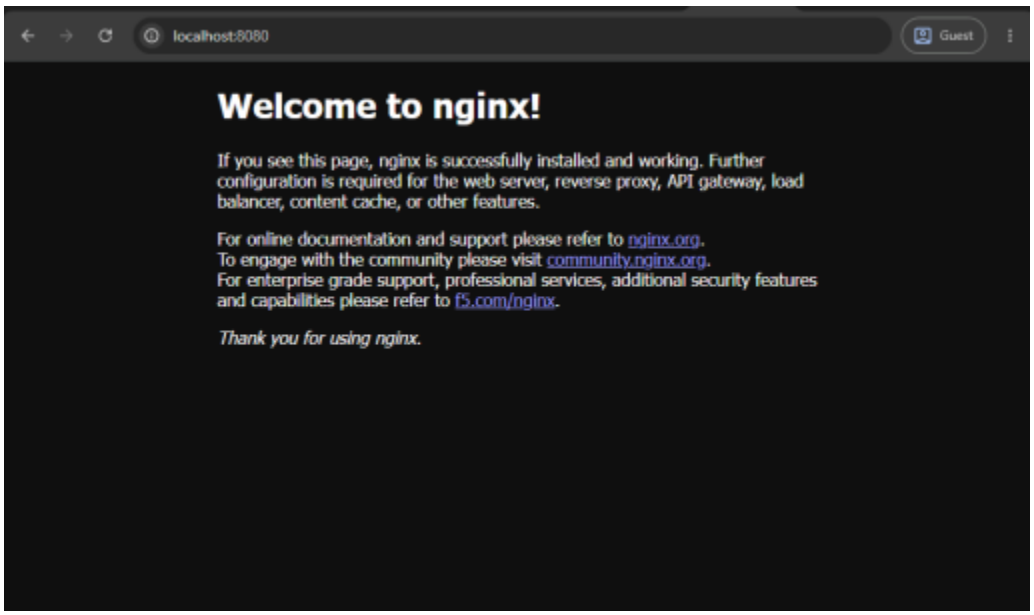
1. Web Server Verification (NGINX):

● URL: http://localhost:8080

● Observation: The default NGINX welcome page successfully loaded in the browser.

● Explanation: This confirms that the webserver container is running properly,

listening on internal port 80, and correctly mapped to host port 8080.



2. Database GUI Verification (PhpMyAdmin):

● URL: http://localhost:8082

● Observation: The PhpMyAdmin login page loaded successfully.

● Credentials Used:

○ Username: student

○ Password: student123

● Explanation: This verifies that the phpmyadmin container is running on port 8082 and successfully communicating over the internal Docker network with the mysqlserver container running on port 3307.

3. Verify Redis (In-Memory Cache):

```
C:\Users\15L\compose-demo>docker exec -it rediscache redis-cli ping
PONG
```

Conclusion - What I learned

● Container Orchestration: Gained hands-on experience using Docker Compose to manage multiple interdependent microservices as a single stack using a single docker-compose.yml configuration file.

● Port Conflict Resolution: Learnt how to resolve host network port conflicts by remapping host ports (e.g., mapping host 3307 to container 3306, and host 8082 to container 80) without altering the internal container configurations.

● Data Persistence: Understood how Docker Volumes (mysql-data) allow database

state to persist independently of the container lifecycle, preventing data loss when containers are recreated or stopped.

● Container Networking & Inter-service Communication: Experienced how Docker automatically establishes an internal network (compose-demo_default), allowing containers like phpmyadmin to talk directly to database using their service names.

● Lifecycle Management: Mastered essential Docker Compose CLI commands for starting (up -d), inspecting (ps), logging (logs), and tearing down (down) multi-container environments.